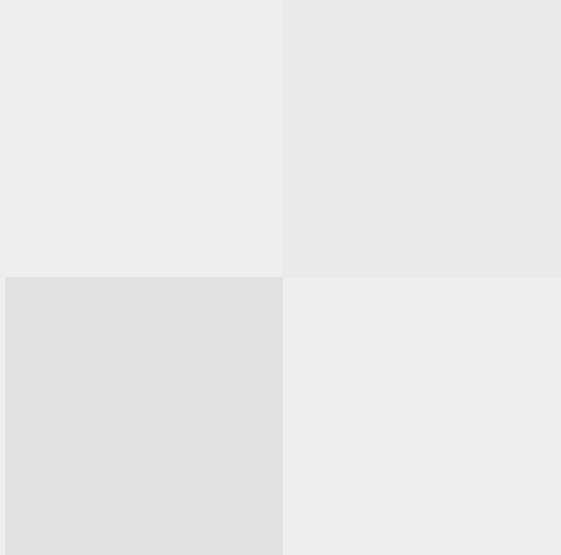
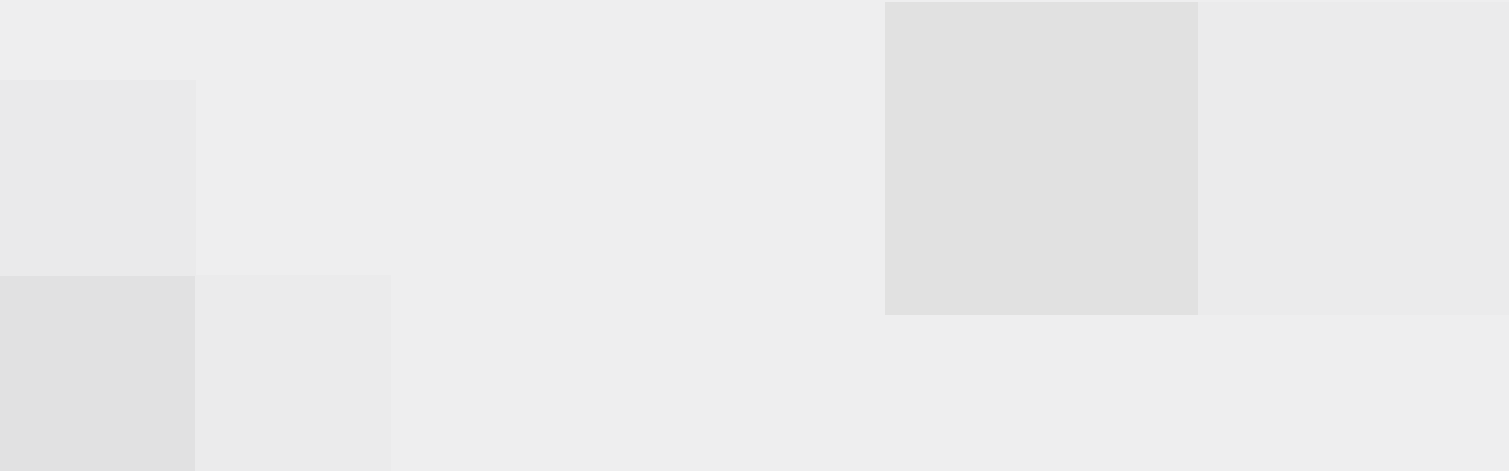




Zespół Szkół Komunikacji im. Hipolita Cegielskiego

Interaktywna symulacja ruchu cząstki w polu magnetycznym

Dominika Bomba i Nikodem Jokiel



Spis treści

1. Cel projektu

2. Specyfikacja projektu

3. Użyte technologie

4. Fizyka i algorytmika

5. Realizacja założeń projektu

6. Umiejętności pozaszkolne

7. Własna ocena

8. Prezentacja działania aplikacji

Cel projektu

- **Nauka**
- **Symulacja fizyczna**
- **Ruch cząsteczki w polu magnetycznym**

Specyfikacja

01

Symulacja fizyczna

Ruch cząstki w polu magnetycznym w czasie rzeczywistym

02

Wizualizacja 2D

Rysowanie trajektorii oraz osi

03

Interfejs i sterowanie

Zmiana parametrów w ImGui - Start, pauza, reset, restart

04

Tryb gry

Dobór parametrów tak, aby trafić w losowy punkt

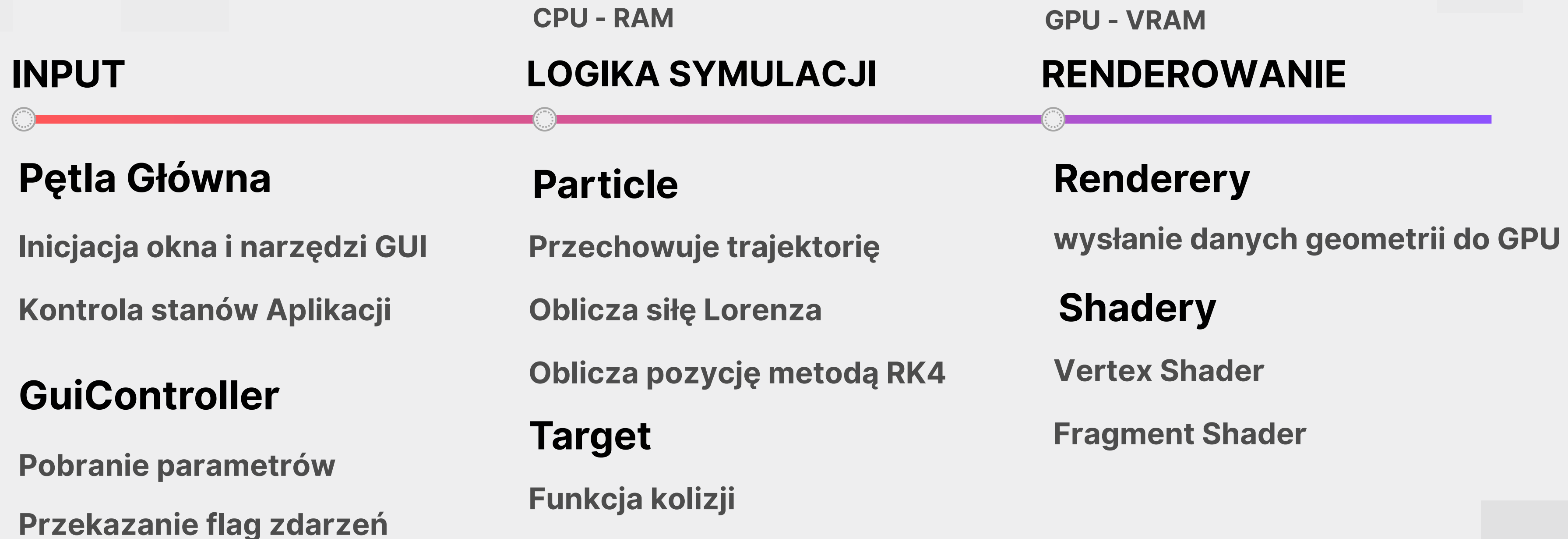
Powody wyboru C++

- **Wysoka wydajność**
- **Pełna kontrola nad pamięcią**
- **Dobra integracja z OpenGL**
- **Dostępność bibliotek wspierających obliczenia i rendering**
- **Stabilność i przenośność**

Dodatkowe wybrane technologie

- **OpenGL** – rendering 2D, wizualizacja
- **Dear ImGui** – interfejs
- **GLFW + GLAD** – okno aplikacji
- **GLM** – wektory i macierze
- **CMake** – zarządzanie kompilacją

Przepływ danych



Architektura kodu



STANY APLIKACJI

Symulacja

Użytkownik swobodnie modyfikuje parametry

Gra

Cel: trafienie do losowanego punktu

FLAGI

simulate

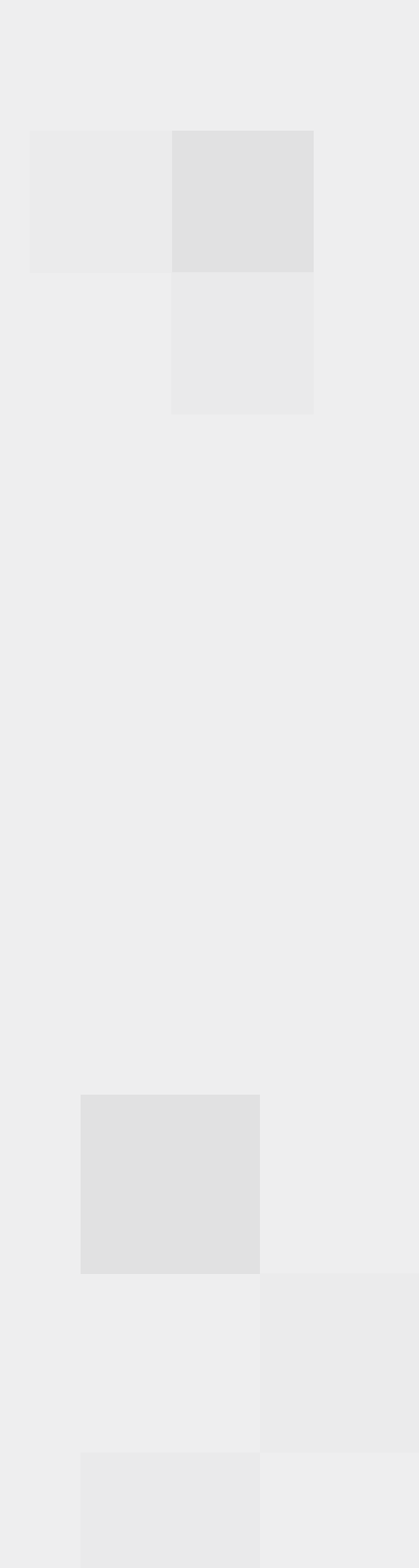
Blokada/wznowienie logiki fizycznej

isHit

Powoduje zatrzymanie symulacji

gameMode

Przełączająca wygląd tła w Fragment Shader





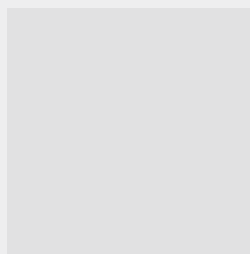
Architektura kodu

PODZIAŁ ZADAŃ

Klasy

KOMPOZYCJA

Klasa Shader



Użyta fizyka

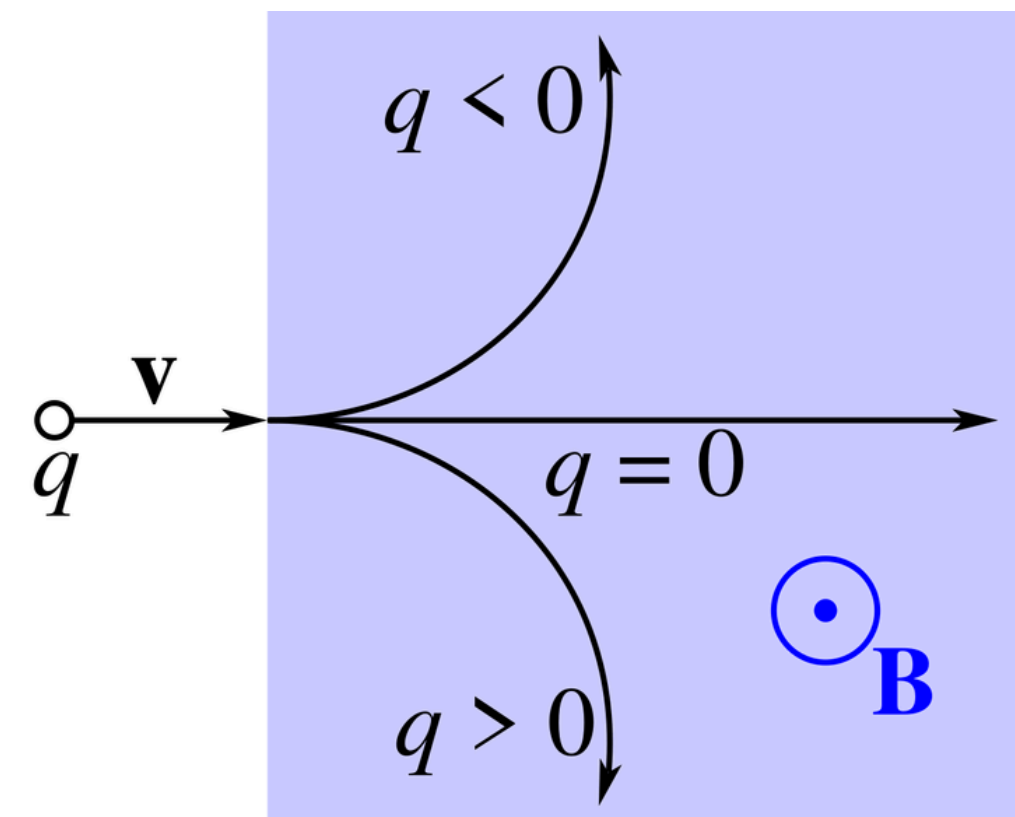
Siła Lorentza

$$\vec{F} = q(\vec{v} \times \vec{B})$$

W układzie 2D (pole wzdłuż osi Z):

$$F_x = q v_y B_z$$

$$F_y = -q v_x B_z$$



Użyta fizyka

Równania ruchu cząstki

$$\frac{dx}{dt} = v_x$$

$$\frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = \frac{q}{m} v_y B_z$$

$$\frac{dv_y}{dt} = -\frac{q}{m} v_x B_z$$

prędkość nie zmienia się pod względem wartości, tylko kierunek się obraca.

Użyta fizyka

Metoda Runge–Kutta 4. rzędu (RK4)

stan po czasie dt

$$y_{n+1} = y_n + \frac{dt}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

aktualny stan cząstki

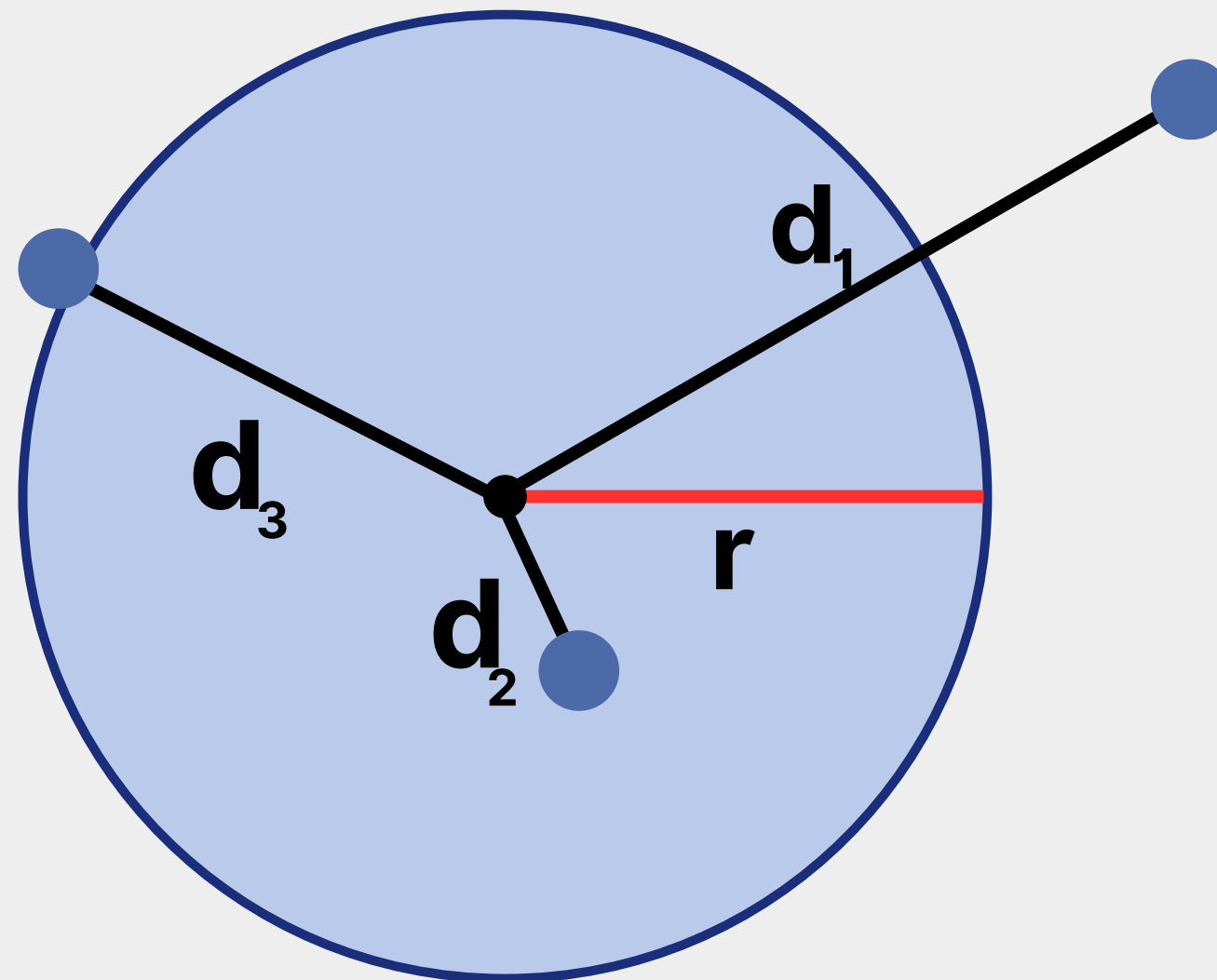
próbki pochodnej

$$\begin{cases} k_1 = f(y_n) \\ k_2 = f\left(y_n + \frac{dt}{2}k_1\right) \\ k_3 = f\left(y_n + \frac{dt}{2}k_2\right) \\ k_4 = f\left(y_n + dt k_3\right) \end{cases}$$

Użyte wzory

Liczenie kwadratów odległości

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad \sqrt{dx^2 + dy^2} \leq r \longrightarrow dx^2 + dy^2 \leq r^2$$

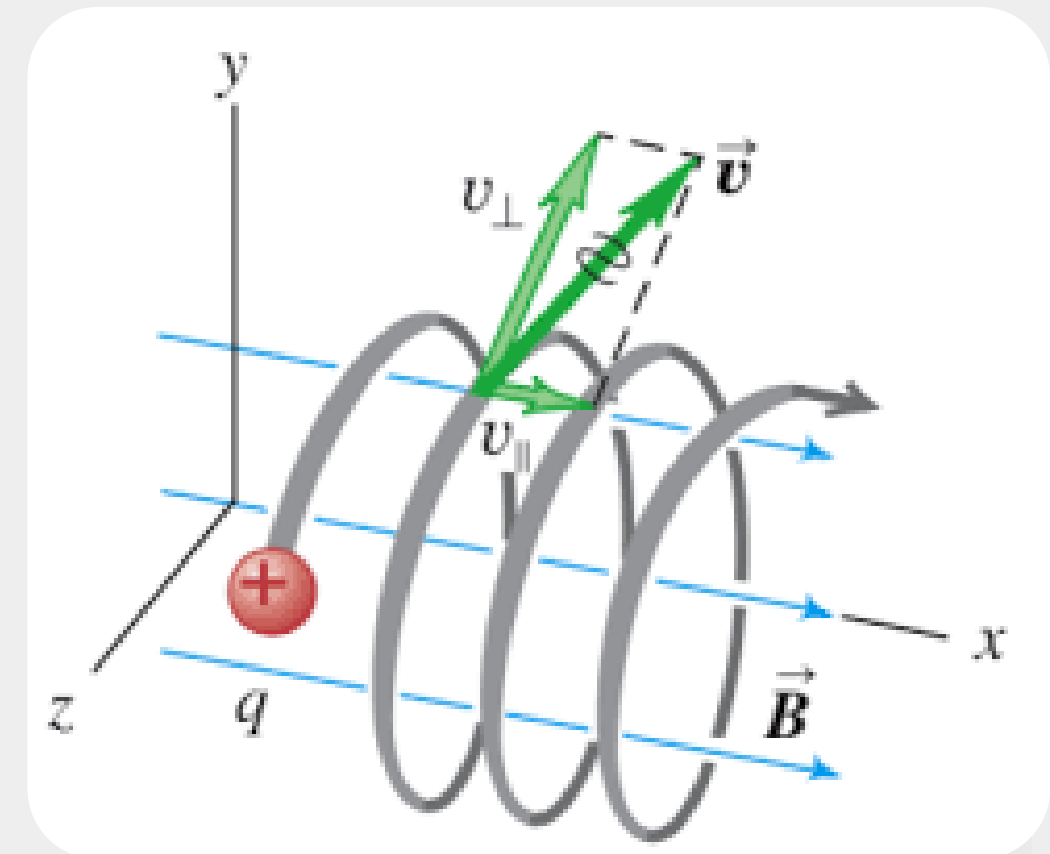


Zrealizowane cele ze specyfikacji

- ✓ **Symulacja 2D**
- ✓ **Tryb gry**
- ✓ **Użyte wzory**
- ✓ **Obliczenia w jednostkach SI**
- ✓ **Plansza z osiami i podziałką**

Nie zrealizowane cele ze specyfikacji

- ✗ Numeryczna skala na osi
- ✗ Siatka układu współrzędnych
- ✗ Parametr kąta startowego



Dodane elementy podczas tworzenia

- + **Skala widocznego obszaru**
- + **Obliczenia kolizji**
- + **Wyświetlanie aktualnego promienia**

Pozaszkolne umiejętności

Rendering 2D w OpenGL

- VAO,
- VBO,
- shadery
- buforowanie punktów trajektorii

Pozaszkolne umiejętności

Obliczenia

- **Metoda Runge–Kutta 4. rzędu (RK4)**
- **Obliczenia wektorowe i macierzowe (GLM)**

Dalszy rozwój

Rozszerzenie Modelu Fizycznego

Definicja pola magnetycznego jako trójwymiarowego wektora

Przebudowa renderowania

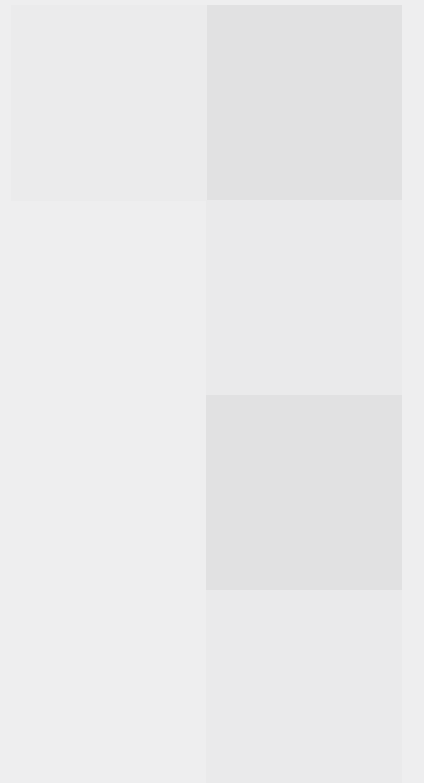
Renderowanie Trajektorii w Przestrzeni

Rozwój interakcji i wizualizacji pola

Kontrola Kamerą

Obiekty 3D

2D → 3D



Nasza ocena projektu

- ✓ **zaawansowana fizyka i użyte algorytmy**
- ✓ **spełnione kryteria ukończenia projektu**
- ✓ **zadania zrealizowane według założonego podziału**